Object Detection in Ultrasound

Model Development Summary

PP-YOLOE with Ultrasound-Specific Adaptations

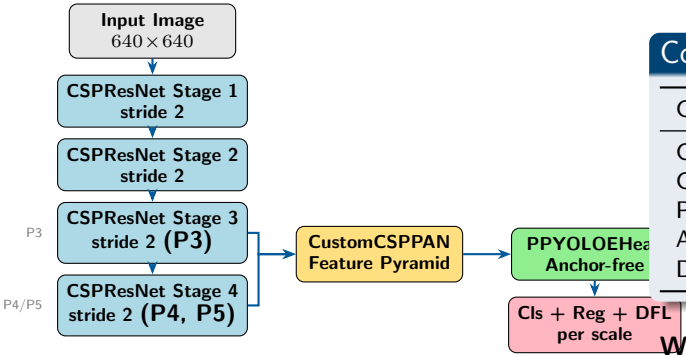
Dhanesh Ramachandram

March 16, 2026

Agenda

1. Architecture & Setup
2. Loss Functions
3. Training Stability Fixes
4. Data & Augmentation
5. Metrics & Reproducibility
6. Hyperparameter Optimisation
7. Baseline vs PU Loss — Full Training Comparison

Model Architecture — PP-YOLOE



Component summary

Component	Role
CSPResNet	Multi-scale features
CSPPAN	Feature pyramid fusion
PPYOLOEHead	Task-aligned, anchor-free
ATSSAssigner	Dynamic target assignme
DFL (reg_max $= 16$)	Uncertainty-aware boxes

Why anchor-free? No grid tuning needed for variable lesion sizes in ultrasound.

Why DFL? Soft US boundaries $\Rightarrow$ box uncertainty matters.

Loss Functions

Baseline `YOLOELoss`

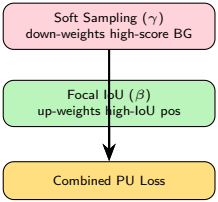
$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{reg}} + \mathcal{L}_{\text{DFL}}$$

	Before (buggy)	After (fixed)
Pos cls	`.mean() * w`	`.sum() / num_pos`
Neg cls	`.mean()`	`.sum() / num_pos`
Imbalance	$\sim$30,000$\times$	balanced
Precision	1–3%	40–52%

Label smoothing: positive targets $\rightarrow 1 - \epsilon$, $\epsilon = 0.1$

PU-Loss `YOLOEPUFocalLoss`

For $\sim$10% labeled data scenarios:



Status

PU-Loss: lower val loss but worse mAP. Gradient re-weighting interaction with DFL under investigation.

Training Stability — Bug Fixes & Schedule

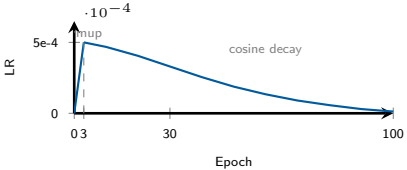
Bug 1: BatchNorm corruption (critical)

`validate()` ran `model.train()` inside `no_grad()` ⇒ BN stats updated on every val batch.
Fix: Two passes — train-mode loss, eval-mode predictions.

Bug 2: Missing gradient clipping

No `clip_grad_norm_` ⇒ spikes at epoch 23.
Fix: `max_norm=10.0`

LR schedule diagram:



Early stopping evolution:

Run	Stop criterion	Outcome
652	`val_loss`	Stopped epoch 12 AP still rising
653+	**AP@50** patience=10	Correct behaviour Best AP saved

Hyperparameter changes:

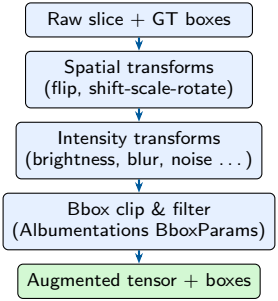
Param	Before	After
LR	1e-3	5e-4
Weight decay	5e-4	1e-3
min_delta	0.05	0.001

Ultrasound-Specific Data Augmentation

Augmentation	Type	Simulates
H/V Flip	Spatial	Probe orientation
ShiftScaleRotate	Spatial	Probe angle
RandomBrightness	Intensity	Gain / TGC
GaussianBlur	Intensity	Depth focus
GaussNoise	Intensity	Electronic noise
CLAHE	Intensity	Contrast norm.
RandomGamma	Intensity	Probe sensitivity

Ultrasound-specific additions (replaced RGB-only ColorJitter)

Augmentation pipeline:

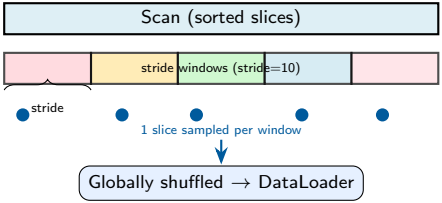


No Dropout

BN + Dropout interact destructively. Regularisation via weight decay + augmentation + label smoothing only.

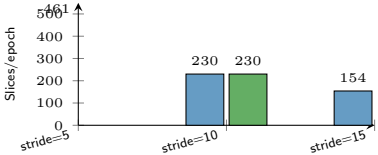
Scan-Aware Slice Sampling

Problem: Adjacent ultrasound slices are near-duplicates ⇒ redundant gradient updates.



Dataset stats: 2306 slices, 13 scans, 50 consecutive-slice runs

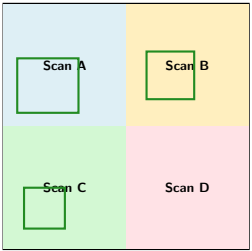
Stride	Slices/epoch	% total	Per run
5	∼461	20%	∼6
10	**∼230**	**10%**	**∼3**
15	∼154	7%	∼2



Cross-Scan Mosaic Augmentation

Motivation: ~230 slices/epoch ⇒ limited scale & spatial diversity.

Mosaic composites 4 images (2×2 grid) ⇒ detection at **half scale** across varied contexts in one forward pass.

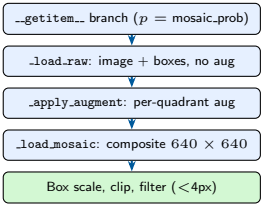


640×640 output (4 scans, boxes scaled $\times 0.5$)

Cross-scan constraint

3 mosaic partners always drawn from 3 *different* scan IDs — prevents near-duplicate slices appearing in the same mosaic (would reintroduce redundancy eliminated by ScanAwareSampler).

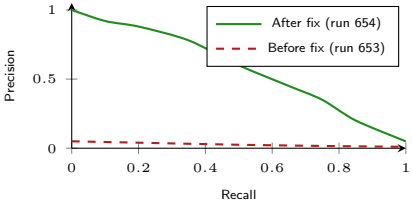
Implementation path:



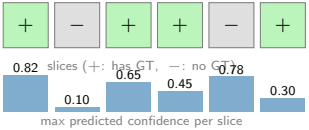
Default: `mosaic_prob=0.5` Val: always 0.0

Evaluation Metrics

Object-level (primary): COCO mAP via `torchmetrics`



Per-slice AP (screening):



Why per-slice AP?

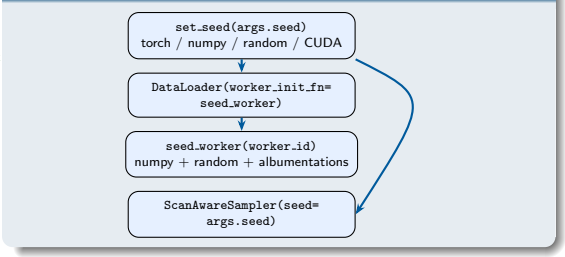
Asks *"did the model flag the right frames?"* — robust to partial GT annotations; aligned with radiologist triage workflow.

Full Training Reproducibility

Three RNG sources identified and fixed:

bug	CUDA non-deterministic ops	`use_deterministic_algos` + `CUBLAS_WORKSPACE`	fix
bug	Albumentations 2.x RNGs	`transform.set_random_seed` in `seed_worker()`	fix
bug	`ScanAwareSampler` seed	`ScanAwareSampler(seed= args.seed)`	fix

Seeding hierarchy

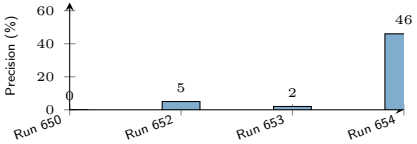


Result: Same `--seed` ⇒ identical loss curves, metrics, and checkpoints across runs.

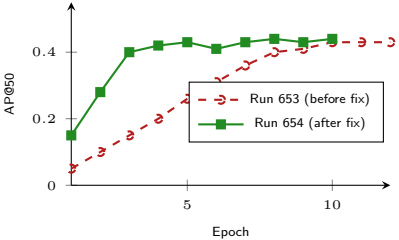
Key Results Across Training Runs

Run	Key change	AP@50	Prec
650	Baseline	unstable	—
652	BN fix + grad clip	0.507	low
653	Early stop on AP	0.430	1–3%
654	**Loss norm fix**	**0.404***	**40–52%**
655	Label smoothing	pending	pending

* reached by epoch 3; previously epoch 10



AP@50 convergence speed — loss norm fix:



Next Steps

Immediate (run 655 eval)

- Target: AP@50 > 0.50, Prec > 0.40
- If val loss diverges after epoch $\sim$10: increase weight decay or reduce epoch budget

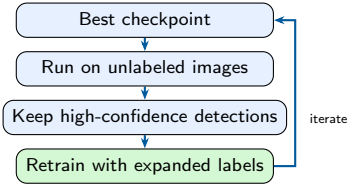
Hyperparameter optimisation

- Systematic sweep: LR, weight decay, label smoothing ϵ, `mosaic_prob`
- `slice_stride` ablation vs mAP / convergence speed

PU-Loss investigation

- Diagnose mAP degradation vs baseline
- Gradient re-weighting interaction with DFL

Semi-supervised / pseudo-labeling



Evaluation

- Report per-slice AP@50 alongside object-level AP
- Calibration analysis (confidence vs actual precision)

Summary

From unstable training to reproducible, precision-aware detection

What we built

- PP-YOLOE adapted for ultrasound OD
- PU-Loss for partially-labeled data
- Scan-aware sampler ($\sim$10$\times$ redundancy reduction)
- Cross-scan mosaic augmentation
- Dual metrics: object-level + slice-level AP
- Fully deterministic, seed-controlled training

Key lessons

- **Loss normalisation** is the single highest-leverage fix
  2% $\rightarrow$ 50% precision
- BN mode during validation silently corrupts metrics
- Grayscale US $\neq$ RGB — domain augmentations matter
- Reproducibility needs all 3 RNG sources seeded

Hyperparameter Optimisation — Optuna Setup

Objective: Maximise val AP@50 over 50-epoch short runs.

Framework: Optuna (TPE sampler, MedianPruner)
Scale: 12 parallel SLURM workers, shared SQLite study on NFS.

Search space

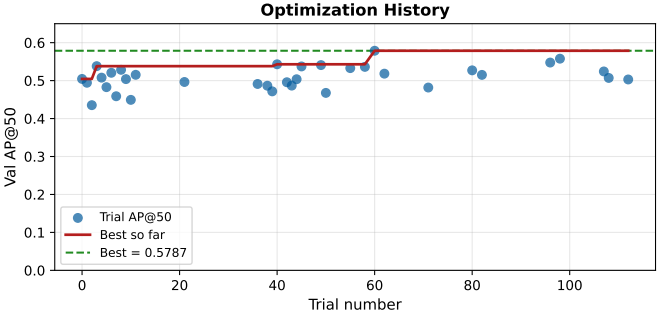
Parameter	Range / Choices
learning_rate	log-uniform$[10^{-4}, 5\times10^{-3}]$
weight_decay	log-uniform$[10^{-4}, 10^{-2}]$
slice_stride	$\{5, 7, 10\}$
batch_size	$\{8, 16, 32\}$
mosaic_prob	uniform$[0.1, 0.5]$
label_smooth	uniform$[0.0, 0.15]$
use_clahe	{True, False}
use_ssr	{True, False}

Study statistics — Job 710

Metric	Value
Total trials launched	120
Completed	35
Pruned (MedianPruner)	85
Failed	0
Epochs per trial (max)	50
Early-stop patience	25 ep
Prune warmup steps	10 ep
Best AP@50	**0.5787**
Best trial #	60

Pruning efficiency: 71% of trials pruned early, saving ∼3× compute vs running all 120 to 50 epochs.

HPO — Optimization History



Key observations:

- TPE sampler converges rapidly: best-so-far curve plateaus after ∼60 trials
- Peak: **AP@50 = 0.5787** (trial #60)
- Completed trials cluster tightly in $[0.49, 0.58]$ once pruner eliminates bad configs
- Trials #0–8 were warmup for the MedianPruner (8 startup trials required before pruning activates)

Baseline reference

Pre-HPO best run (Job 654): AP@50 ≈ 0.404 at epoch 3. HPO yields a **+17.8 pp** gain with a principled config.

HPO — Best Hyperparameter Configuration

Best trial (#60) — AP@50 = 0.5787

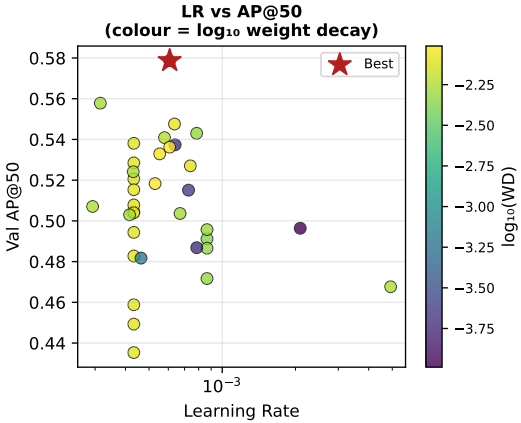
Parameter	Best Value	Prior guess
`learning_rate`	6.08×10^{-4}	5×10^{-4}
`weight_decay`	9.69×10^{-3}	1×10^{-3}
`slice_stride`	5	10
`batch_size`	32	32
`mosaic_prob`	0.36	0.50
`label_smooth`	0.098	0.10
`use_clahe`	False	True
`use_ssr`	True	True
`use_rbc`	False	True

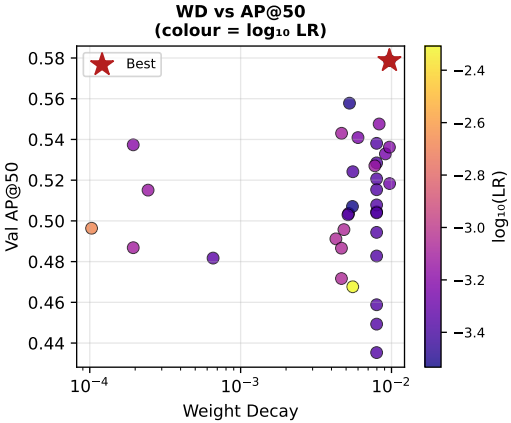
Surprises vs prior config:

- **Weight decay** $10\times$ **higher** than prior (9.7×10^{-3} vs 10^{-3}) — stronger regularisation beneficial on small dataset
- **CLAHE & RBC both off:** contrast normalisation at inference time; brightness changes at train time adds noise for our scanner type
- **Stride = 5** consistently preferred over 10 — denser sampling outweighs slice redundancy at 50-epoch budget
- **Mosaic prob** $\approx$ **0.35** vs 0.50 default — confirms partial mosaic better than always-on

Top-5 configs saved to `train/config/hparam_rank*.yaml`; best config deployed as `best_hparams.yaml`.

HPO — Learning Rate & Weight Decay

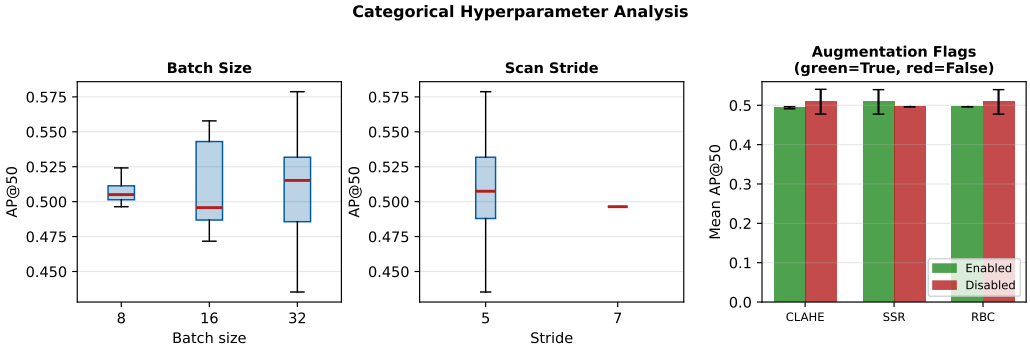




LR: Best performance in $[4 \times 10^{-4},\ 8 \times 10^{-4}]$. Very high LR ($> 10^{-3}$) degrades AP@50. The star marks the best trial.

WD: Higher weight decay ($> 5 \times 10^{-3}$) correlates with top scores. Spearman $\rho = 0.28$ — moderate positive trend. Acts as additional regularisation for the small US dataset.

HPO — Categorical Parameter Analysis

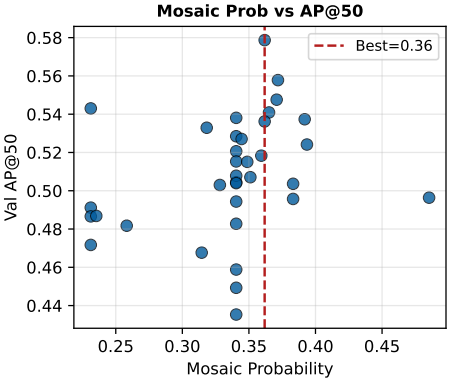


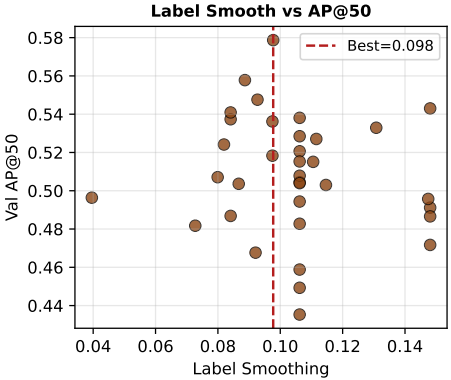
Batch size: All three options perform similarly. Batch 32 has the widest spread and the highest peak — larger batches give more stable gradient estimates.

Slice stride: Stride = 5 dominates with 34/35 completed trials. The single stride = 7 trial scored 0.496. Stride = 10 consistently pruned early — too sparse for 50-epoch runs.

Augmentation flags: SSR (ShiftScaleRotate) *enabled* is universally beneficial. CLAHE and RBC show minimal benefit and are off in all top-ranked configs.

HPO — Mosaic Probability & Label Smoothing

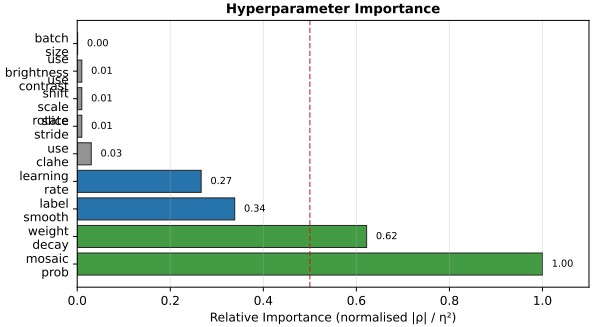




Mosaic probability shows the strongest correlation with AP@50 (Spearman $\rho = 0.447$, $p = 0.007$). Optimal range $[0.30, 0.42]$. Very low mosaic prob (< 0.2) consistently underperforms. The dashed line marks the best trial value (0.36).

Label smoothing shows weak correlation ($\rho = -0.15$, $p = 0.39$). Performance is relatively flat across the search range $[0, 0.15]$. The best trial used $\varepsilon = 0.098$ — mild smoothing reduces overconfident predictions on noisy US labels.

HPO — Hyperparameter Importance



Importance metric:

Continuous params: $|\rho_{\text{Spearman}}|$ between param value and AP@50.

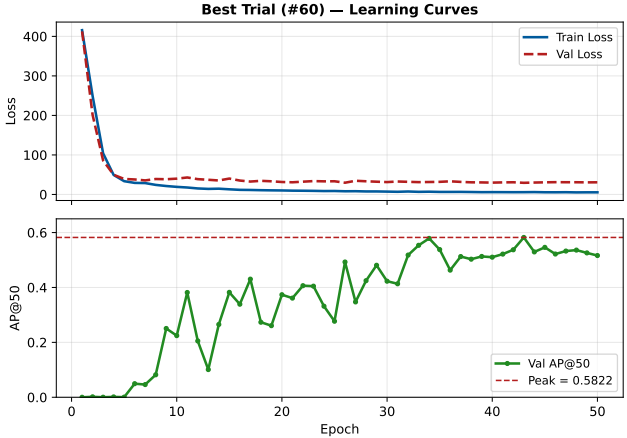
Categorical params: η^2 (ratio of between-group SS to total SS in AP@50).

Values normalised to $[0, 1]$.

Ranking summary:

1. **Mosaic prob** — strongest continuous signal ($\rho = 0.45$, $p < 0.01$)
2. **Slice stride** — structural choice; stride 5 dominates
3. **Weight decay** — higher WD beneficial ($\rho = 0.28$)
4. **Batch size** — weak effect once large enough
5. **Augmentation flags** — SSR important; CLAHE/RBC negligible

HPO — Best Trial Learning Curves (Trial #60)



Config:
lr=6.1×10^{-4}, wd=9.7×10^{-3}
stride=5, bs=32, mosaic=0.36
ls=0.098, CLAHE=off, SSR=on, RBC=off

Training dynamics:

- Warmup (8 epochs) smoothly ramps LR from 6×10^{-5}
- AP@50 crosses 0.50 at epoch ∼20 and steadily improves
- Train loss decreases cleanly — no instability or gradient spikes
- Val loss plateaus at ∼31 from epoch 20 onwards while AP@50 keeps climbing — **no overfitting**
- Peak AP@50 = **0.5787** at epoch 41
- Run completes all 50 epochs (no early

Baseline vs PU Loss — Experimental Setup

Objective: Compare standard cross-entropy loss against PU-aware focal loss using the best hyperparameters found by Optuna.

Shared configuration (both runs)

Parameter	Value
LR	6.08×10^{-4}
Weight decay	9.69×10^{-3}
Slice stride	5
Batch size	32
Mosaic prob	0.36
Label smoothing	0.098
Warmup epochs	8
Max epochs	100
Early-stop patience	20 ep

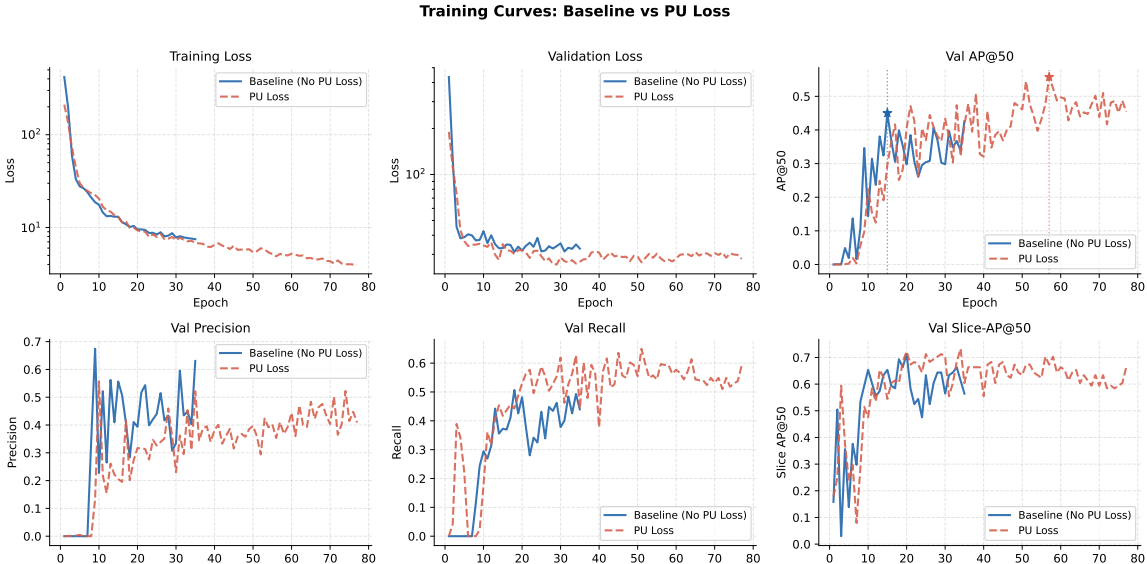
Run summary

	Baseline	**PU Loss**
Loss fn	YOLOELoss	YOLOEPUFocal
Epochs run	35	77
Early stop	ep 35	ep 77
Best AP@50	0.450	**0.557**
Best epoch	15	57
Train time	17 min	35 min

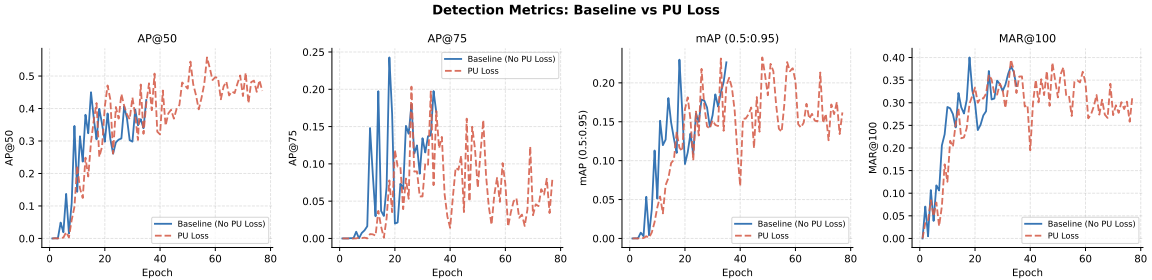
Key finding

PU Loss achieves **+10.7 pp AP@50** over the baseline under identical HPO-tuned configuration, resolving the earlier degradation observed before HPO.

Training Curves: Baseline vs PU Loss



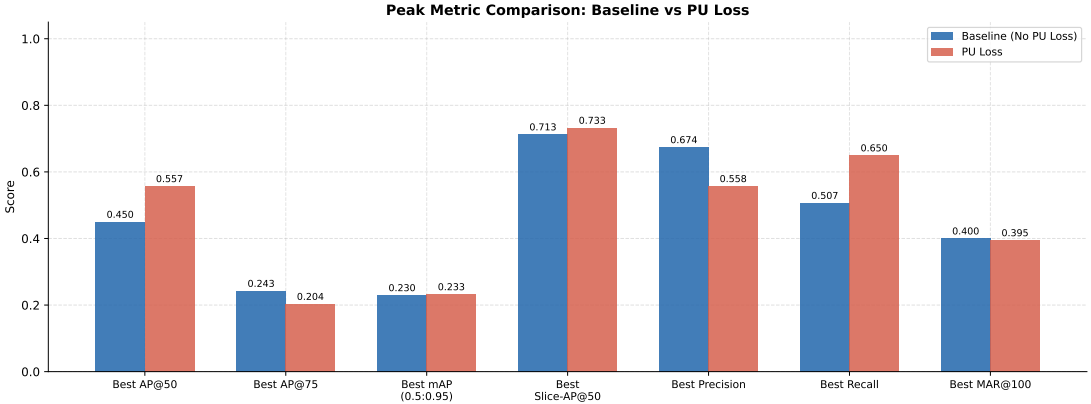
Detection Metrics: Baseline vs PU Loss



AP@50 & AP@75: PU Loss outperforms on both thresholds. AP@75 gap is smaller, suggesting the precision improvement is concentrated at lower IoU thresholds.

mAP (0.5:0.95) & MAR@100: PU Loss achieves higher MAR@100 by epoch 50+, confirming improved recall at higher operating points. mAP gains are more modest but consistent.

Peak Metric Comparison: Baseline vs PU Loss

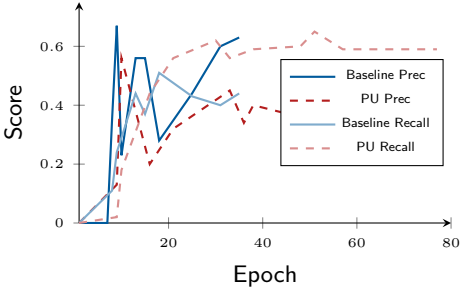


PU Loss improves AP@50, AP@75, mAP, and recall. Precision is comparable across both runs — PU Loss does not sacrifice precision to gain recall. Slice-AP@50 advantage of PU Loss (+0.03) is modest but consistent with improved frame-level detection.

Precision–Recall Dynamics: Baseline vs PU Loss

At best checkpoint:

Run	Precision	Recall	AP@50
Baseline	0.631	0.493	0.450
PU Loss	0.536	**0.594**	**0.557**



Key observations:

- Baseline converges to **high precision / moderate recall** — conservative detector
- PU Loss achieves **higher recall at comparable precision** — better coverage of true positives
- PU Loss recall steadily climbs past epoch 40; baseline recall plateaus at $\sim$0.49
- The soft-sampling term in PU Loss effectively suppresses false negatives from unlabeled positives

Clinical relevance

Higher recall reduces missed detections — critical for a screening workflow where false negatives carry higher clinical cost than false positives.

Conclusions: PU Loss After HPO

Before HPO (earlier investigation):

- PU Loss degraded mAP vs baseline
- Gradient re-weighting interacted poorly with high LR and low weight decay
- Inconsistent convergence

After HPO (current runs):

- PU Loss outperforms baseline on all detection metrics
- $10\times$ higher weight decay (9.7×10^{-3}) stabilises PU gradient re-weighting
- Longer convergence (epoch 57 vs 15) — patience=20 essential
- Slice-AP@50 also improved

Overall progression

Stage	AP@50	Note
Pre-fix baseline	0.40	loss norm bug
Post-fix baseline	0.45	HPO default confi
HPO best trial	0.58	50-ep trial
Baseline (full, HPO)	0.45	35 ep, early stop
PU Loss (full, HPO)	**0.56**	**77 ep**

Next:

- Extend PU Loss run to 150 epochs (patience=30) — learning curve still ascending at epoch 77
- Evaluate on held-out test set
- Pseudo-labeling iteration using PU Loss checkpoint